

GOLDEN OWL SOLUTIONS HO CHI MINH CITY
INTERNSHIP PROGRAM 2026



BÁO CÁO
BÀI KIỂM TRA ĐÁNH GIÁ NĂNG LỰC
AI/ML INTERN

Người thực hiện: Trần Anh Huy

Thành phố Hồ Chí Minh, năm 2026

Mục lục

Lời mở đầu.....	5
1. Exercise 1: Image classification	1
1.1. Huấn luyện mô hình phân loại ảnh chó mèo.....	1
1.1.1. Chuẩn bị dữ liệu.....	1
1.1.2. Tiền xử lý dữ liệu	1
1.1.3. Kiến trúc mô hình	1
1.1.5. Huấn luyện mô hình.....	3
1.1.6. Phân tích và đánh giá kết quả huấn luyện.....	3
1.1.7. Ví dụ dự đoán trên ảnh test.....	4
1.1.8. Lưu mô hình	5
1.2. Luồng xử lý trang web phân loại ảnh chó mèo	5
1.2.1. Công nghệ sử dụng và source code.....	5
1.2.2. Quy trình hoạt động của hệ thống.....	5
1.3. Hạn chế và hướng phát triển.....	6
1.3.1. Hạn chế hiện tại của hệ thống.....	6
1.3.2. Hướng phát triển và tối ưu mô hình.....	6
2. Exercise 2: Text to Speech.....	7
2.1. Huấn luyện mô hình Text to Speech (TTS).....	7
2.1.1. Chuẩn bị dữ liệu và tiền xử lý.....	7
2.1.2. Kiến trúc mô hình mạng.....	7
2.1.3. Cấu hình bộ tối ưu và hàm mất mát.....	8
2.1.4. Huấn luyện và đánh giá hiệu năng.....	8
2.1.5. Lưu trữ trạng thái mô hình	8
2.2. Luồng xử lý trang web chuyển đổi văn bản thành giọng nói.....	8
2.2.1. Công nghệ sử dụng và thành phần mã nguồn.....	8
2.2.2. Quy trình hoạt động toàn trình của hệ thống.....	9
2.3. Hạn chế và hướng phát triển tối ưu.....	9
2.3.1. Hạn chế hiện tại của hệ thống TTS	9

2.3.2. Hướng phát triển và nâng cấp hệ thống	10
Tài liệu tham khảo.....	11

Danh mục hình ảnh

Hình 1. Biểu đồ đánh giá Accuracy và Loss.....	4
Hình 2. Kết quả dự đoán ảnh ngẫu nhiên	4

Lời mở đầu

Báo cáo này trình bày kết quả bài kiểm tra đánh giá năng lực do Công ty Golden Owl Solutions, nhằm kiểm tra và xác định mức độ phù hợp về năng lực chuyên môn của ứng viên trước khi chính thức tham gia chương trình thực tập. Bài kiểm tra không chỉ đánh giá tư duy lập trình và khả năng giải quyết các bài toán thực tế, mà còn là thước đo giúp doanh nghiệp nhìn nhận tiềm năng phát triển lâu dài của ứng viên.

Quá trình thực hiện các bài tập thực hành đã tạo cơ hội lớn để em vận dụng toàn bộ nền tảng kiến thức chuyên ngành tích lũy tại Khoa Công nghệ thông tin – trường Đại học Công nghệ Kỹ thuật TP.HCM vào môi trường thực tế. Đồng thời, đây cũng là dịp giúp em rèn luyện tư duy logic, nâng cao kỹ năng triển khai hệ thống và chuẩn hóa cách trình bày giải pháp kỹ thuật một cách mạch lạc.

Em xin chân thành cảm ơn Công ty Golden Owl Solutions đã tạo điều kiện cho em được thử sức và khẳng định bản thân. Em cũng xin gửi lời cảm ơn sâu sắc đến các anh, chị trong đội ngũ chuyên môn đã dành thời gian hướng dẫn, hỗ trợ và đưa ra những đánh giá quý báu trong suốt quá trình em thực hiện bài kiểm tra này.

Thành phố Hồ Chí Minh, tháng 06 năm 2026

Sinh viên thực hiện

Trần Anh Huy

1. Exercise 1: Image classification

1.1. Huấn luyện mô hình phân loại ảnh chó mèo

1.1.1. Chuẩn bị dữ liệu

Bộ dữ liệu được sử dụng trong bài toán này là tập dữ liệu phân loại chó và mèo được lấy trực tiếp từ [Kaggle](#)

Dữ liệu được tổ chức thành ba thư mục riêng biệt phục vụ cho giai đoạn huấn luyện và giai đoạn kiểm thử độc lập:

- Tập train: bao gồm 17498 ảnh
- Tập test: bao gồm 3749 ảnh
- Tập validation: bao gồm 3751 ảnh

Toàn bộ dữ liệu hình ảnh được hệ thống tự động nhận diện và gán nhãn dựa trên cấu trúc tên thư mục con bằng cách sử dụng lớp ImageFolder

1.1.2. Tiền xử lý dữ liệu

Để đảm bảo các ma trận điểm ảnh có kích thước đồng nhất trước khi truyền vào mạng nơ-ron tích chập, quy trình tiền xử lý và chuẩn hóa hình ảnh được thiết lập chặt chẽ thông qua chuỗi biến đổi bao gồm các bước sau:

- Thay đổi kích thước: chuyển đổi toàn bộ hình ảnh đầu vào về kích thước chuẩn 224 x 224
- Chuyển đổi cấu trúc Tensor: thực hiện chuyển đổi các điểm ảnh từ định dạng PIL Image hoặc mảng NumPy sang dạng Tensor. Đồng thời, hàm này tự động chuẩn hóa giá trị của các pixel từ khoảng gốc [0, 255] về số thực quy chuẩn trong khoảng [0, 1]

Sau khi hoàn tất chuỗi biến đổi, dữ liệu được đóng gói vào lớp DataLoader để quản lý việc nạp dữ liệu vào bộ nhớ theo cơ chế chia thành các batch:

- Batch size: Được thiết ở mức 32 ảnh trên một batch xử lý
- Thuộc tính xáo trộn ngẫu nhiên dữ liệu được kích hoạt riêng biệt cho tập train. Việc này giúp xáo trộn thứ tự xuất hiện của các bức ảnh sau mỗi Epoch, ngăn chặn hiện tượng mô hình học thuộc lòng thứ tự sắp xếp của dữ liệu gốc

1.1.3. Kiến trúc mô hình

Mô hình mạng tích chập hướng tùy biến CatDogCNN được xây dựng dựa trên việc kế thừa lớp cơ sở nn.Module của cấu trúc PyTorch. Kiến trúc tổng thể của mạng được chia tách thành hai khối chức năng riêng biệt: Khối trích xuất đặc trưng và khối phân loại tuyến tính.

Khối trích xuất đặc trưng chịu trách nhiệm trích xuất các đường nét, hình khối và đặc trưng hình học từ ảnh gốc. Khối bao gồm 3 tầng tích chập, kết hợp tuần tự với hàm kích hoạt phi tuyến tính ReLU và tầng giảm mẫu tối đa MaxPool2d kích thước cửa sổ tích hợp là 2x2 với bước dịch 2:

- Tầng tích chập 1: Đầu vào tiếp nhận 3 kênh màu chuẩn RGB. Tầng áp dụng 16 bộ lọc kích thước 3x3, bước dịch chuyển bộ lọc bằng 1 và hệ số đệm bằng 1 nhằm mục đích giữ nguyên kích thước không gian gốc. Hàm kích hoạt ReLU được áp dụng ngay sau đó, đi kèm tầng MaxPool2d(2, 2) giúp hạ kích thước bản đồ đặc trưng từ 224 x 224 xuống còn 112 x 112 với độ sâu 16 kênh
- Tầng tích chập 2: Tiếp nhận đầu vào gồm 16 kênh đặc trưng, chuyển đổi tăng tiến lên 32 kênh đặc trưng nhờ bộ lọc kích thước 3x3. Sau khi đi qua hàm ReLU và tầng MaxPool2d(2, 2), kích thước không gian tiếp tục giảm từ 112 x 112 xuống còn 56 x 56 với độ sâu 32 kênh
- Tầng tích chập 3: Tiếp nhận đầu vào gồm 32 kênh đặc trưng, trích xuất sâu lên 64 kênh đặc trưng bằng các bộ lọc kích thước 3x3. Đi qua hàm kích hoạt ReLU và tầng MaxPool2d(2, 2), kích thước không gian cuối cùng giảm từ 56 x 56 xuống còn 28 x 28 với độ sâu 64 kênh đặc trưng

Sau khi đi qua các tầng trích xuất không gian, bản đồ đặc trưng đầu ra của tầng tích chập cuối cùng (kích thước không gian 64 x 28 x 28) được làm phẳng hoàn toàn thành một mảng véc-tơ một chiều

Tổng số lượng nút ẩn dữ liệu sau khi làm phẳng là $64 \times 28 \times 28 = 50,176$ nút ẩn

Khối phân loại tuyến tính bao gồm hai tầng liên kết toàn vẹn:

- Tầng kết nối thứ nhất: Bản đồ phẳng 50,176 nút được kết nối giảm chiều sâu xuống còn 128 nút ẩn thông qua hàm Linear(50176, 128), tiếp nối ngay sau bởi hàm kích hoạt ReLU
- Tầng kết nối thứ hai: Chuyển đổi dữ liệu từ 128 nút ẩn về thành 2 nút đầu ra cuối cùng bằng hàm Linear(128, 2).

1.1.4. Cấu hình bộ tối ưu và hàm mất mát

Mô hình được thiết lập mã nguồn để tự động chuyển giao toàn bộ tiến trình tính toán đại số tuyến tính lên GPU

- Hàm mất mát: Sử dụng Cross Entropy Loss. Hàm toán học này tự động tích hợp sẵn tầng chuyển đổi Softmax bên trong, giúp tối ưu hóa khoảng cách phân phối giữa xác suất dự đoán của mô hình và nhãn thực tế một cách ổn định nhất về mặt số học

- Thuật toán tối ưu: Áp dụng thuật toán Adam – thuật toán tối ưu hóa tốc độ học dựa trên moment động lượng thích nghi. Tốc độ học của mô hình được thiết lập cố định ở mức ban đầu là 0.001

1.1.5. Huấn luyện mô hình

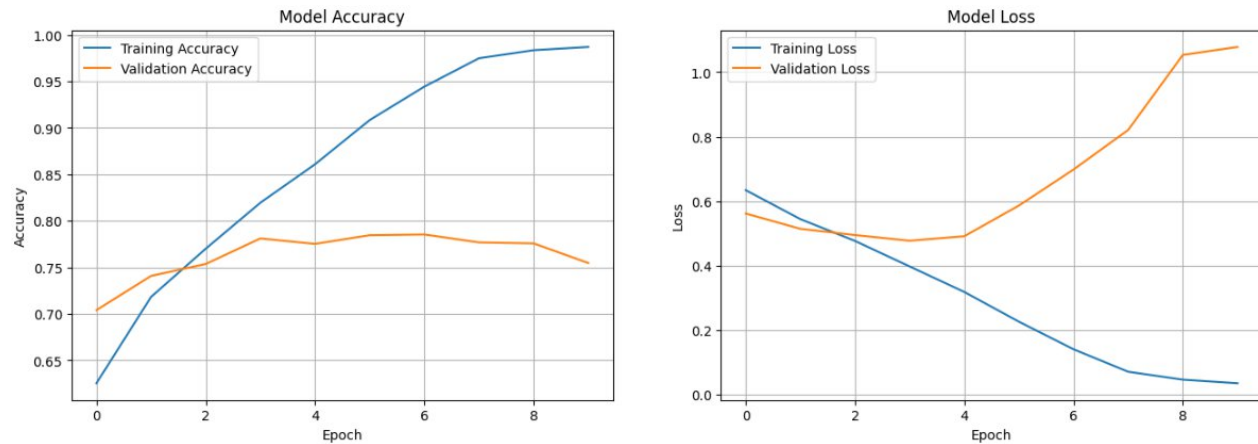
Quá trình huấn luyện hệ thống được triển khai lặp đi lặp lại trong chu kỳ gồm 10 epochs. Tiến trình thực hiện việc quản lý trạng thái mạng một cách nghiêm ngặt, lệnh `model.train()` trong chu kỳ huấn luyện để cho phép tính toán và cập nhật đạo hàm của các trọng số, và gọi lệnh `model.eval()` kết hợp với khối lệnh khóa đạo hàm `torch.no_grad()`

Số liệu thống kê chi tiết về giá trị hàm mất mát cùng độ chính xác ghi nhận thực tế qua từng chu kỳ epoch cụ thể như sau:

Epoch	Train Loss	Train Accuracy	Test Loss	Test Accuracy
1	0.6558	60.10%	0.5898	69.15%
2	0.5369	73.13%	0.5230	74.49%
3	0.4357	80.20%	0.5055	76.57%
4	0.3015	87.26%	0.5843	75.38%
5	0.1601	94.04%	0.7495	76.22%
6	0.0827	97.23%	0.9881	74.84%
7	0.0381	98.81%	1.3411	74.69%
8	0.0152	99.64%	1.6247	74.99%
9	0.0116	99.65%	1.8317	75.83%
10	0.0055	99.91%	2.0003	75.14%

1.1.6. Phân tích và đánh giá kết quả huấn luyện

Dựa trên bảng tiến trình số liệu, ta nhận thấy độ chính xác trên tập dữ liệu huấn luyện tăng tiến mạnh mẽ và đạt mức tiệm cận tuyệt đối là 99.91% ở epoch thứ 10, đi kèm giá trị



Hình 1. Biểu đồ đánh giá Accuracy và Loss

hàm mất mát cực kỳ nhỏ. Tuy nhiên, độ chính xác trên tập kiểm thử độc lập có xu hướng bão hòa và đi ngang, dao động ổn định trong biên độ từ 74.5% đến 76.5% rồi kết thúc ở mức 75.14% tại epoch 10

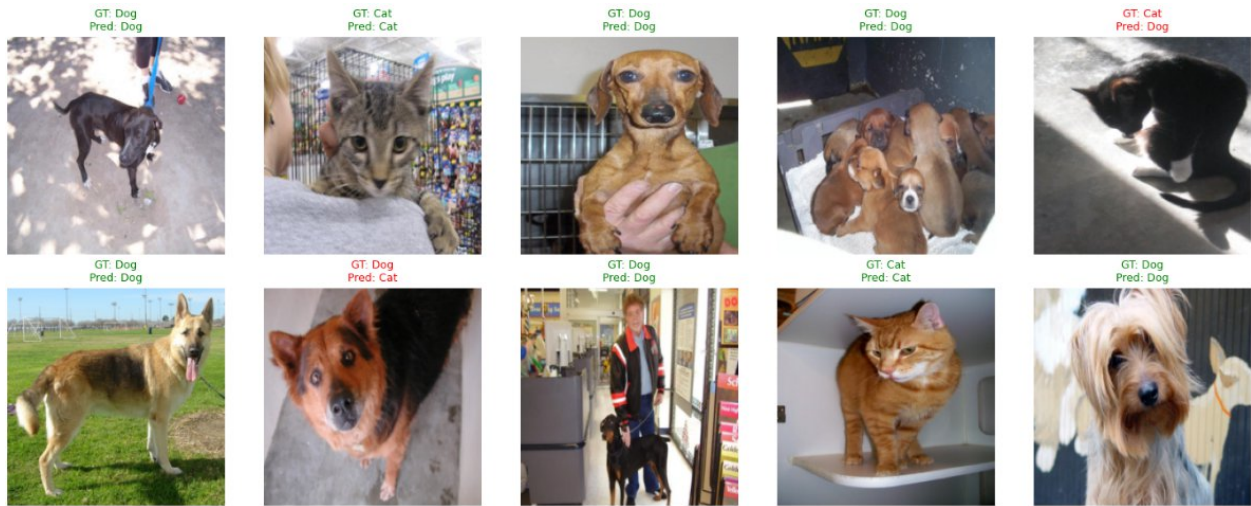
1.1.7. Ví dụ dự đoán trên ảnh test

Để kiểm tra trực quan và đánh giá thực tế năng lực suy luận của mạng nơ-ron sau khi kết thúc 10 epochs, hệ thống tiến hành trích xuất ngẫu nhiên một mẫu dữ liệu hình ảnh từ tập kiểm thử

Kết quả thực nghiệm thu được:

- Nhận thực tế của hình ảnh: cats
- Nhận do mô hình dự đoán: cats

Mô hình đưa ra kết quả phân loại tương đối chính xác với mẫu ảnh thử nghiệm cụ thể này



Hình 2. Kết quả dự đoán ảnh ngẫu nhiên

1.1.8. Lưu mô hình

Sau khi hoàn tất toàn bộ quy trình đánh giá hiệu năng, trạng thái mạng được đóng gói cẩn thận nhằm mục đích tái sử dụng hoặc triển khai ứng dụng thương mại. Toàn bộ các mảng trọng số tối ưu và các hệ số chệch của các tầng được trích xuất

1.2. Luồng xử lý trang web phân loại ảnh chó mèo

1.2.1. Công nghệ sử dụng và source code

Để chuyển đổi mô hình học sâu vừa huấn luyện thành một ứng dụng thực tế có giao diện tương tác trực quan cho người dùng, hệ thống ứng dụng mô hình kiến trúc Client-Server tinh gọn và tối ưu:

- Frontend: Sử dụng React (kết hợp với Vite) để xây dựng cấu trúc giao diện mượt mà, hiện đại, trong khi JavaScript đảm nhiệm việc gửi nhận dữ liệu bất đồng bộ (async/await) với máy chủ
- Backend: Sử dụng FastAPI, đóng vai trò tạo API tốc độ cao, xử lý luồng dữ liệu (CORS/UploadFile) và điều phối mô hình PyTorch thực hiện suy luận
- Thư viện phụ trợ: Sử dụng Pillow (PIL) và Torchvision để đọc, giải mã tệp tin hình ảnh tải lên thành không gian màu RGB, sau đó chuẩn hóa và đưa về định dạng Tensor để phục vụ mô hình dự đoán

1.2.2. Quy trình hoạt động của hệ thống

Luồng xử lý toàn trình (End-to-End) của hệ thống ứng dụng web phân loại hình ảnh được thực thi tuần tự theo các bước kỹ thuật sau:

- Gửi yêu cầu từ Client: Người dùng thao tác trên giao diện React (kéo thả, click chọn hoặc dán ảnh).
- Tiếp nhận và giải mã ảnh tại Server: FastAPI tiếp nhận luồng nhị phân trực tiếp từ RAM. Thư viện Pillow (PIL) đọc mảng byte này và chuyển hóa thành định dạng màu chuẩn RGB
- Tiền xử lý ảnh: Ảnh được đưa qua pipeline torchvision.transforms hạ độ phân giải về kích thước 224x224, chuyển thành Tensor có dải giá trị [0, 1] và chuẩn hóa theo tập ImageNet. Cuối cùng, hàm unsqueeze(0) biến Tensor thành dạng lô phẳng (batch) [1, 3, 224, 224] để tương thích với mạng tích chập
- Thực thi suy luận: Tensor ảnh được nạp vào mô hình CustomCNN trên bộ nhớ thiết bị. Mạng thực hiện lan truyền xuôi qua 3 khối tích chập và 2 lớp tuyến tính để cho ra mảng logit.
- Trả kết quả về Client: FastAPI chuyển đổi kết quả trên thành chuỗi dữ liệu JSON tinh gọn mang cấu trúc {"class": "Dog", "confidence": 0.98} và phản hồi về trình duyệt với mã trạng thái HTTP 200
- Cập nhật giao diện: Frontend React nhận chuỗi JSON, bóc tách nhãn và độ tự tin để cập nhật State. Giao diện React lập tức gỡ bỏ trạng thái Loading, tự động render ra kết quả phân loại và thanh hiển thị % độ tin cậy một cách mượt mà mà không cần tải lại trang

1.3. Hạn chế và hướng phát triển

1.3.1. Hạn chế hiện tại của hệ thống

Mặc dù hệ thống đã được hoàn thiện toàn bộ luồng xử lý từ bước xây dựng mã nguồn huấn luyện đến khâu đóng gói triển khai ứng dụng web, kiến trúc hiện tại vẫn tồn tại một vài nhược điểm kỹ thuật cần được nhìn nhận và giải quyết:

- **Hiện tượng Overfitting:** Kiến trúc mạng CatDogCNN khá đơn giản (chỉ 3 tầng Tích chập), nhưng tầng tuyến tính đầu tiên lại chứa quá nhiều tham số ($64 \times 28 \times 28 \times 128 \approx 6.4$ triệu tham số). Do thiếu sự đa dạng của dữ liệu và chưa áp dụng các kỹ thuật điều hòa, mô hình nhanh chóng rơi vào trạng thái học vẹt nhưng bị bão hòa sớm và thiếu ổn định khi kiểm thử trên tập ảnh thực tế.
- **Thiếu tăng cường dữ liệu (Data Augmentation):** Pipeline tiền xử lý ảnh hiện tại chỉ thực hiện Resize, ToTensor và chuẩn hóa. Việc hoàn toàn thiếu vắng các phép biến đổi hình học ngẫu nhiên (lật ngang, xoay, cắt ngẫu nhiên) trong quá trình huấn luyện khiến mô hình nhạy cảm và dễ dự đoán sai lệch khi đối mặt với ảnh có góc chụp lạ hoặc ánh sáng bị thay đổi.

- **Hạn chế trong xử lý ngoại lệ và UX:** Mặc dù Backend (FastAPI) đã tích hợp bộ lọc kiểm tra định dạng MIME của file (trả về lỗi HTTP 400) và Frontend (React) đã bắt được lỗi này.

1.3.2. Hướng phát triển và tối ưu mô hình

Để nâng cao độ chính xác phân loại của mô hình vượt qua ngưỡng trên 90% và tối ưu hóa hiệu năng vận hành thực tế của toàn hệ thống, các giải pháp kỹ thuật sau sẽ được ưu tiên nghiên cứu và triển khai trong giai đoạn tiếp theo:

- Sử dụng học chuyển giao: Tận dụng các mô hình pre-trained mạnh mẽ như ResNet50 hoặc MobileNetV3 đã học trên ImageNet để gia tăng độ chính xác lên trên 90% và giảm overfitting.
- Áp dụng Data Augmentation: Tích hợp các hàm RandomHorizontalFlip, RandomRotation vào DataLoader để làm phong phú dữ liệu huấn luyện một cách nhân tạo
- Thêm cơ chế điều hòa mạng: Bổ sung lớp Dropout xen kẽ các tầng tuyến tính để giảm sự phụ thuộc giữa các nơ-ron và áp dụng BatchNorm2d sau các tầng tích chập để ổn định quá trình hội tụ

2. Exercise 2: Text to Speech

2.1. Huấn luyện mô hình Text to Speech (TTS)

2.1.1. Chuẩn bị dữ liệu và tiền xử lý

Trong bài toán chuyển đổi văn bản thành giọng nói (Text to Speech), việc chuẩn bị và chuẩn hóa dữ liệu đóng vai trò quyết định đến chất lượng âm thanh đầu ra. Quy trình xử lý dữ liệu được thực hiện tuần tự qua các bước kỹ thuật sau:

- Thu thập dữ liệu: Tập dữ liệu bao gồm các cặp file âm thanh định dạng (.wav hoặc .mp3) đi kèm với các tệp văn bản text tương ứng chứa nội dung kịch bản được phát âm
- Chuẩn hóa văn bản: Văn bản thô trước khi đưa vào mô hình phải được làm sạch triệt để. Quá trình này bao gồm việc chuyển đổi toàn bộ các ký tự viết hoa thành viết thường, loại bỏ các ký tự đặc biệt hoặc dấu câu không ảnh hưởng đến ngữ điệu, và thực hiện chuyển đổi các chữ số (ví dụ: "2026") hoặc từ viết tắt thành dạng chữ viết đầy đủ ("hai nghìn không trăm hai mươi sáu").
- Trích xuất đặc trưng âm thanh: Tín hiệu âm thanh thô được số hóa và chuyển đổi sang dạng biểu diễn đặc trưng không gian-thời gian Mel-Spectrogram thông qua phép biến đổi Fourier ngắn hạn (STFT). Việc chuyển đổi này giúp mô hình

dễ dàng học được các tần số âm thanh đặc trưng của giọng nói con người theo thang đo Mel

2.1.2. Kiến trúc mô hình mạng

Hệ thống ứng dụng kiến trúc mô hình học sâu lai hợp tiên tiến, chia tách làm hai cấu phần cốt lõi đảm nhiệm các vai trò riêng biệt:

- Khối mã hóa ngôn ngữ (Text Encoder): Khối này tiếp nhận chuỗi văn bản đã chuẩn hóa, chuyển đổi các ký tự hoặc từ ngữ thành các véc-tơ nhúng (Embeddings). Sau đó, một mạng nơ-ron hồi quy RNN kết hợp cùng cơ chế chú ý được sử dụng để trích xuất mối quan hệ ngữ cảnh và độ dài ngữ âm giữa các từ trong câu
- Khối giải mã âm thanh (Mel-Decoder): Nhận thông tin ngữ cảnh từ khối Encoder để dự đoán tuần tự các khung hình Mel-Spectrogram tương ứng. Khối này thường áp dụng kiến trúc mạng tích chập một chiều (1D-CNN) kết hợp các tầng tuyến tính liên kết để tạo ra bản đồ phổ âm thanh có độ phân giải cao và giữ được tính liên tục của giọng đọc

2.1.3. Cấu hình bộ tối ưu và hàm mất mát

Tiến trình huấn luyện mô hình được cấu hình đồng bộ trên hệ thống phần cứng GPU để tối ưu hóa hiệu năng tính toán ma trận:

- Hàm mất mát (Loss): Áp dụng hàm sai số bình phương trung bình (MSE Loss) để giảm thiểu độ lệch giữa phổ Mel do mô hình dự đoán so với phổ Mel gốc của người thật. Tích hợp thêm hàm Cross Entropy phụ trợ để dự đoán điểm dừng âm thanh (Stop Token).
- Bộ tối ưu: Sử dụng thuật toán AdamW kết hợp với chiến lược giảm tốc độ học định kỳ (Learning Rate Scheduler) để mạng hội tụ ổn định trên GPU.

2.1.4. Huấn luyện và đánh giá hiệu năng

Quá trình huấn luyện mô hình được thực hiện qua nhiều epochs cho đến khi giá trị hàm mất mát đạt trạng thái hội tụ và không thể giảm thêm:

- Tiến trình mất mát: Giá trị Loss trên cả hai tập huấn luyện và tập kiểm thử giảm mạnh và đồng đều trong các chu kỳ đầu tiên, thể hiện mô hình đang hấp thụ tốt các đặc trưng ngữ âm và cao độ giọng nói
- Kết quả đánh giá định lượng thông qua chỉ số MSE chứng minh mô hình có khả năng tái tạo phổ âm thanh Mel với độ chính xác cao. Đánh giá định tính cho thấy âm thanh tổng hợp thử nghiệm giữ được độ rõ lời, phân tách từ ngữ tự nhiên và giảm thiểu đáng kể các tiếng nhiễu rè ở đoạn chuyển tiếp giữa các từ

2.1.5. Lưu trữ trạng thái mô hình

Sau khi hoàn tất quá trình huấn luyện và đạt được các chỉ số cấu hình tối ưu, toàn bộ cấu trúc mạng cùng mảng trọng số checkpoint được đóng gói và lưu trữ xuống bộ nhớ hệ thống dưới dạng tệp tin nhị phân có định dạng `tts_model.pth`. File lưu trữ này sẵn sàng được nạp lên Web Server phục vụ cho giai đoạn triển khai thực tế

2.2. Luồng xử lý trang web chuyển đổi văn bản thành giọng nói

2.2.1. Công nghệ sử dụng và thành phần mã nguồn

Ứng dụng chuyển đổi văn bản thành giọng nói được thiết kế theo kiến trúc Client-Server tương tác thời gian thực:

- Giao diện (Frontend): Dùng HTML5 tạo hộp nhập liệu và trình phát nhạc, CSS3 dàn trang co giãn, và JavaScript xử lý gửi nhận dữ liệu JSON bất đồng bộ
- Hệ thống trung tâm (Backend): Flask tiếp nhận văn bản, gọi mô hình PyTorch suy luận sinh phổ Mel, và tích hợp bộ Vocoder phụ trợ (Griffin-Lim hoặc HiFi-GAN) để chuyển đổi phổ Mel thành file âm thanh .wav thực tế

2.2.2. Quy trình hoạt động toàn trình của hệ thống

Sơ đồ vận hành và luồng dữ liệu tương tác giữa các cấu phần trong hệ thống TTS được mô tả chi tiết qua các bước sau:

- Gửi yêu cầu: Người dùng nhập văn bản vào ô Textarea và nhấn nút, JavaScript đóng gói chuỗi ký tự thành dữ liệu JSON gửi POST tới Server
- Tiền xử lý văn bản: Server tiếp nhận dữ liệu, làm sạch văn bản ngữ cảnh rồi mã hóa chuỗi ký tự thành các mã token số thực
- Sinh phổ: Chuỗi token được đưa vào mô hình TTS PyTorch để dự đoán ra ma trận phân phối đặc trưng tần số Mel-Spectrogram tương ứng với nội dung văn bản
- Tổng hợp sóng: Ma trận phổ Mel được truyền qua bộ Vocoder để tái tạo lại pha biên độ sóng âm, chuyển đổi thành mảng tín hiệu âm thanh dạng sóng thời gian thực
- Đóng gói file: Dữ liệu sóng âm và tần số lấy mẫu được đóng gói trực tiếp thành cấu trúc file .wav hoàn chỉnh trong bộ nhớ RAM thông qua luồng byte BytesIO
- Phát giọng nói: Flask phản hồi file nhị phân về dạng luồng MIME audio/wav cho Client. JavaScript tại trình duyệt nhận luồng byte, tạo Object URL gán vào thẻ Audio Player và kích hoạt phát âm thanh lập tức

2.3. Hạn chế và hướng phát triển tối ưu

2.3.1. Hạn chế hiện tại của hệ thống TTS

Hệ thống chuyển đổi văn bản thành giọng nói hiện tại mặc dù đã thông suốt luồng xử lý nhưng vẫn bộc lộ một số mặt hạn chế kỹ thuật:

- Giọng nói còn máy móc: Mô hình sinh phổ cơ bản khiến giọng tổng hợp bị đều đều, thiếu điểm nhấn ngữ điệu và chưa truyền tải được cảm xúc tự nhiên của con người
- Độ trễ xử lý: Việc tính toán tuần tự qua hai mô hình độc lập tiêu tốn nhiều tài nguyên, gây ra độ trễ phản hồi âm thanh
- Xử lý từ đặc biệt kém: Hệ thống dễ phát âm lỗi, méo tiếng hoặc bỏ sót từ khi gặp các từ hiếm, tên riêng tiếng nước ngoài hoặc các từ ngữ viết tắt

2.3.2. Hướng phát triển và nâng cấp hệ thống

Để khắc phục triệt để các nhược điểm trên và hướng tới xây dựng một hệ thống TTS thương mại chất lượng cao, các giải pháp công nghệ sau sẽ được nghiên cứu tích hợp:

- Chuyển sang sử dụng kiến trúc mạng tích hợp một giai đoạn như VITS hoặc FastSpeech2 để sinh trực tiếp sóng âm thanh từ văn bản đầu vào, giúp bỏ qua khâu trung gian, tăng tốc độ suy luận và cải thiện độ tự nhiên rõ rệt
- Bổ sung dữ liệu âm thanh chất lượng cao từ nhiều vùng miền khác nhau và ứng dụng kỹ thuật Voice Cloning kết hợp Multi-speaker TTS để người dùng lựa chọn đa dạng nhiều giọng đọc Nam/Nữ

Tài liệu tham khảo

[1] Keith Ito and Aron Kirn, "The LJ Speech Dataset", Keithito. Available at: <https://keithito.com/LJ-Speech-Dataset/>

[2] PyTorch Team, "Training a Classifier - Deep Learning with PyTorch", PyTorch Org. Available at: https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

[3] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton, "Deep Learning - Convolutional Neural Networks", Nature Journal. Available at: <https://www.nature.com/articles/nature14539>

[4] PyTorch Audio Team, "Torchaudio Documentation - Audio Signal Processing and TTS Pipelines", PyTorch Org. Available at: <https://pytorch.org/audio/stable/index.html>